

Ad Launch → Render Migration Guide

This guide covers deploying the Ad Launch frontend to [Render](https://render.com) (<https://render.com>) as a Docker web service, linked to the GitHub repo `wyatt-lgtm/ad-launch`.

1. Files Added

File	Purpose
<code>nextjs_space/Dockerfile</code>	Multi-stage Docker build (deps → build → production runner)
<code>nextjs_space/.dockerignore</code>	Keeps the Docker context small
<code>render.yaml</code>	Render Blueprint — auto-configures the web service
<code>nextjs_space/.env.render.example</code>	Template for all required environment variables

2. Quick Start

Option A: Blueprint (recommended)

1. Push all files to `wyatt-lgtm/ad-launch` `main` branch.
2. Go to [Render Dashboard](https://dashboard.render.com/) (<https://dashboard.render.com/>) → **New** → **Blueprint**.
3. Connect the `wyatt-lgtm/ad-launch` repo.
4. Render reads `render.yaml` and creates the service.
5. Go to the service's **Environment** tab and fill in all `sync: false` variables (see §3).
6. Trigger a manual deploy or push a commit.

Option B: Manual Docker Service

1. Render Dashboard → **New** → **Web Service** → Docker.
 2. Connect the `wyatt-lgtm/ad-launch` repo.
 3. Set **Dockerfile Path** = `./nextjs_space/Dockerfile`
 4. Set **Docker Context** = `./nextjs_space`
 5. Set environment variables (see §3).
 6. Deploy.
-

3. Environment Variables

Copy values from the current `.env` file. Key differences for Render:

Variable	Notes
<code>DATABASE_URL</code>	Same Postgres URL as current deployment. Both Render and Abacus will share the same DB.
<code>NEXTAUTH_SECRET</code>	Must match the current value — existing sessions/tokens depend on it.
<code>NEXTAUTH_URL</code>	Set to your production URL, e.g. <code>https://connect.launchmarketing.com</code>
<code>TOMBSTONE_API_URL</code>	<code>https://tombstone-api-xjc4.onrender.com</code>
<code>AWS_ACCESS_KEY_ID</code>	NEW — the current deployment uses <code>AWS_PROFILE</code> (IAM role). Render needs explicit credentials. Create an IAM user with S3 read/write access to your bucket.
<code>AWS_SECRET_ACCESS_KEY</code>	NEW — see above.
<code>AWS_REGION</code>	Same as current.
<code>AWS_BUCKET_NAME</code>	Same as current.
<code>AWS_FOLDER_PREFIX</code>	Same as current.

4. How the Dockerfile Works

```

Stage 1 (deps):    Install node_modules
Stage 2 (builder): Fix Prisma schema paths → prisma generate → next build (standalone)
Stage 3 (runner):  Copy standalone output + Prisma client → node server.js

```

Key details:

- The Prisma schema has a hardcoded `output` path and `binaryTargets` specific to the Abacus platform. The Dockerfile `sed`s these out during build:
- Removes the absolute `output = "/home/ubuntu/..."` line (uses default `node_modules/.prisma/client`)
- Replaces `binaryTargets` with `["native", "debian-openssl-3.0.x"]` for the Docker runtime
- `NEXT_OUTPUT_MODE=standalone` is set at build time so Next.js produces a self-contained `server.js`
- The `.yarnrc.yml` is overridden to remove platform-specific global cache paths

5. Known Migration Items

5a. Email Notifications (Abacus-specific)

The app currently sends emails via the **Abacus AI notification API** (`sendNotificationEmail`). This API uses platform-specific notification IDs (`NOTIF_ID_*`) and the `WEB_APP_ID` . These **will not work** when running outside Abacus.

Affected files:

- `app/api/register/route.ts` — signup confirmation email
- `app/api/auth/forgot-password/route.ts` — password reset email
- `lib/scout-email.ts` — daily scout report emails
- `app/api/scout/completion-check/route.ts` — post-ready notification
- `app/api/scout/daily-run/route.ts` — daily scout trigger

Migration options:

1. **SendGrid / Resend / AWS SES** — Replace `sendNotificationEmail()` calls with a third-party email provider.
2. **Keep Abacus for email only** — Run the Abacus deployment alongside Render just for email routing. The `NOTIF_ID_*` values will still work if `WEB_APP_ID` and `ABACUSAI_API_KEY` are set.
3. **Disable emails temporarily** — The app will function without email; users just won't receive confirmation/reset emails.

5b. Custom Domain DNS

If moving `connect.launchmarketing.com` to Render:

1. Add the custom domain in Render's service settings.
2. Update DNS to point to Render's load balancer (CNAME or A record).
3. Render handles TLS automatically.
4. Remove the domain from the Abacus deployment.

Important: Don't change DNS until you've verified the Render deployment works.

5c. Health Check

The `render.yaml` uses `/api/auth/providers` as the health check path (returns 200 with JSON). Adjust if needed.

5d. Prisma Migrations

When you add new Prisma schema fields in the future:

- Run `npx prisma db push` from your local machine (or a Render shell) against the production `DATABASE_URL`.
- The Dockerfile runs `prisma generate` at build time but does NOT run migrations/push.

6. Dual-Deployment Period

During migration, you can run **both** the Abacus and Render deployments simultaneously:

- They share the same database, so data is consistent.
- Use Abacus for the existing `ad-launch-1nfyr8.abacusai.app` URL.
- Use Render for `connect.launchmarketing.com` (after DNS switch).
- Once Render is stable, decommission the Abacus deployment.

7. Render Plan Sizing

Plan	RAM	CPU	Monthly	Notes
Starter	512 MB	0.5	\$7	Fine for low traffic; may cold-start
Standard	2 GB	1.0	\$25	Recommended for production
Pro	4 GB	2.0	\$85	If you see memory pressure

The standalone Next.js server typically uses 150-300 MB at idle.